

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«АЛТАЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных технологий
Кафедра информатики

**РАЗРАБОТКА ПЛАГИНА ДЛЯ LMS MOODLE ДЛЯ ГЕНЕРАЦИИ ТЕСТОВЫХ
ЗАДАНИЙ НА ОСНОВЕ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА**
(магистерская диссертация)

Выполнил:
студент 4.306М-3 группы
Федорук Ангелина Дмитриевна

(подпись)

Научный руководитель:
Старший преподаватель
Кротова Ольга Сергеевна

(подпись)

Допустить к защите:
Зав. кафедрой, к.ф.-м.н., доцент
Козлов Денис Юрьевич

(подпись)

«__» _____ 2025 г.

Работа защищена:
«__» _____ 2025 г.

Оценка: _____

Председатель ГЭК:

(подпись)

Барнаул 2025

РЕФЕРАТ

Тема выпускной квалификационной работы: «Разработка плагина для LMS Moodle для генерации тестовых заданий на основе искусственного интеллекта».

Цель работы – разработка плагина для LMS Moodle для генерации тестовых заданий на основе отечественного искусственного интеллекта.

Предмет исследования – методы и технологии интеграции искусственного интеллекта в процесс автоматической генерации учебных заданий, а также средства разработки плагинов на языке программирования PHP для LMS Moodle.

Объект исследования – процесс формирования тестовых заданий в систему управления обучением (LMS), в частности, платформа Moodle.

В результате выпускной квалификационной работы решены следующие задачи: проведен анализ предметной области и существующих решений по автоматической генерации тестовых заданий с использованием отечественного искусственного интеллекта, изучены особенности архитектуры и возможности платформы Moodle для разработки плагинов на языке PHP, исследованы методы интеграции ИИ в образовательные системы, выбраны и адаптированы технологии, подходящие для реализации генерации заданий, разработан и протестирован плагин для Moodle, обеспечивающий формирование тестов на основе заданной темы или области знаний.

Объем работы – 56 страница, количество рисунков – 18, таблиц – 2, приложений – 1, использованных источников – 40.

Ключевые слова: искусственный интеллект, Moodle, генерация тестов, PHP, плагин, образовательные технологии, нейросеть.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ГЕНЕРАЦИИ ТЕСТОВЫХ ЗАДАНИЙ И LMS MOODLE	6
1.1. Обзор систем дистанционного обучения (LMS)	6
1.2. Виды и классификация тестовых заданий в образовательных системах	8
1.3. Подходы к автоматической генерации тестов	10
1.4. Роль искусственного интеллекта в образовательных технологиях	13
1.5. Обзор существующих решений по генерации тестов с использованием ИИ	15
2. ПОСТАНОВКА ЗАДАЧИ И ПРОЕКТИРОВАНИЕ ПЛАГИНА ДЛЯ MOODLE	17
2.1. Анализ требований к плагину	17
2.2. Выбор методов ИИ для генерации тестовых заданий	20
2.3. Выбор технологий разработки	22
2.4. Архитектура и структура разрабатываемого плагина	26
2.5. Проектирование интерфейса пользователя	28
2.6. Обеспечение безопасности и защиты данных	30
3. РАЗРАБОТКА ПЛАГИНА ДЛЯ MOODLE	32
3.1. Разработка плагина	32
3.2. Тестирование плагина	38
3.3. Устранение ошибок и оптимизация работы	43
ЗАКЛЮЧЕНИЕ	44
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	46
ПРИЛОЖЕНИЕ	49

ВВЕДЕНИЕ

Современные образовательные технологии стремительно развиваются, и одним из приоритетных направлений является цифровизация обучения. В условиях массового перехода образовательных учреждений к дистанционным и смешанным формам обучения возрастает значимость систем управления обучением (LMS), среди которых одной из самых популярных является платформа Moodle. Эффективное использование таких систем требует автоматизации рутинных процессов, в частности – создания тестовых заданий, что особенно актуально при работе с большими объемами учебного материала.

Одним из перспективных подходов к решению данной задачи является использование искусственного интеллекта (ИИ). Применение ИИ позволяет не только ускорить процесс подготовки тестов, но и сделать задание более адаптивными и соответствующими уровню знаний обучающегося. Автоматическая генерация заданий с использованием ИИ способствует снижению нагрузки на преподавателя и повышению индивидуализации образовательного процесса.

Целью выпускной квалификационной работы является разработка плагина для платформы Moodle, реализующего генерацию тестовых заданий на основе отечественных технологии искусственного интеллекта.

Разрабатываемый в рамках выпускной квалификационной работы плагин предназначен для упрощения и ускорения процесса составления контрольных и тестовых заданий преподавателями, что особенно актуально при дистанционном обучении и массовых онлайн-курсах. Внедрение ИИ в данный процесс позволяет формировать задания, соответствующие содержанию курса, тем самым повышая эффективность обучения.

Актуальность и практическая значимость выпускной квалификационной работы обусловлены активным развитием цифровых образовательных ресурсов, растущими требованиями к качеству дистанционного обучения, а также широкой распространённостью Moodle в образовательных организациях. Автоматизация создания тестовых заданий с использованием ИИ позволяет не только оптимизировать

деятельность преподавателей, но и внедрять элементы интеллектуальной адаптации в учебный процесс.

Для достижения поставленной цели в работе решаются следующие задачи:

1. Изучение предметной области, связанной с генерацией тестов и применением ИИ в образовании;
2. Изучение архитектуры Moodle и технологий разработки плагинов на языке программирования PHP;
3. Исследование и выбор подходящих моделей искусственного интеллекта для генерации тестовых заданий;
4. Разработка плагина для Moodle с возможностью интеграции искусственного интеллекта.

Выпускная квалификационная работа состоит из введения, трех глав, заключения, библиографического списка и приложений. В введение сформулированы цель и задачи выпускной квалификационной работы, определены объект и предмет исследования, раскрыта актуальность и практическая значимость выпускной работы.

В первой главе представлен теоретический обзор платформы Moodle, рассмотрены технологии, применяемые для разработки плагинов, а также современные подходы к генерации учебных заданий с использованием искусственного интеллекта.

Вторая глава – посвящена анализу существующих решений и проектирование структуры разрабатываемого программного продукта.

Третья глава содержит описание процесса реализации плагина, его тестирования, а также оценку эффективности предложенного решения.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ГЕНЕРАЦИИ ТЕСТОВЫХ ЗАДАНИЙ И LMS MOODLE

1.1. ОБЗОР СИСТЕМ ДИСТАНЦИОННОГО ОБУЧЕНИЯ (LMS)

Системы управления обучением (Learning Management Systems, LMS) представляют собой специализированное программное обеспечение, предназначенное для автоматизации процессов организации, проведения и контроля учебной деятельности в электронном формате. LMS позволяют преподавателям создавать курсы, управлять учебным контентом, отслеживать успеваемость обучающихся, проводить тестирование и собирать статистику об образовательном процессе.

Основными функциональными возможностями LMS являются:

- размещение и структурирование учебных материалов (тексты, презентации, видео и др.);
- управление пользователями и их ролями (преподаватель, студент, администратор);
- организация тестирования, контрольных и экзаменационных заданий;
- сбор и анализ статистических данных об активности и успеваемости обучающихся;
- обеспечение взаимодействия участников образовательного процесса через форумы, чаты, рассылки и другие инструменты коммуникации.

Среди наиболее популярных LMS на мировом рынке можно выделить:

- Moodle – открытая платформа с широкой экосистемой модулей и поддержкой пользовательских расширений;
- Blackboard – коммерческая система, широко применяемая в вузах Европы;
- Canvas – облачная LMS, ориентированная на интуитивно понятный интерфейс и простоту интеграции с внешними сервисами;
- Google Classroom – решение, активно внедряемое в школьное образование благодаря тесной интеграции с другими сервисами Google.

Каждая из систем имеет свои преимущества и ограничения, однако ключевыми требованиями к LMS остаются стабильность работы, масштабируемость, гибкость

настройки и поддержка современных педагогических моделей (например, смешанное обучение).

На отечественном рынке также активно развиваются LMS, ориентированные на специфику российского образования, например:

- «Электронный университет» (на базе 1С),
- «Цифровая школа»,
- платформа «Сферум» и др.

Среди всех перечисленных решений Moodle занимает особое место благодаря следующим характеристикам:

- Открытый исходный код, позволяющий адаптировать систему под конкретные задачи и интегрировать собственные модули;
- Широкая поддержка форматов тестов, включая XML-импорт и экспорт;
- Развитое сообщество разработчиков и пользователей;
- Возможность локализации, в том числе на русском языке;
- Совместимость с различными ИИ-инструментами и внешними API.

В связи с этим Moodle широко используется в российских вузах как основа для реализации цифровых образовательных курсов, а также в рамках дистанционного и смешанного обучения. На сегодняшний день система поддерживает более 150 миллионов пользователей по всему миру и остаётся одним из наиболее гибких и настраиваемых решений для образовательных учреждений.

1.2. ВИДЫ И КЛАССИФИКАЦИЯ ТЕСТОВЫХ ЗАДАНИЙ В ОБРАЗОВАТЕЛЬНЫХ СИСТЕМАХ

Тестирование — одна из ключевых форм контроля знаний в образовательном процессе, широко применяемая в электронном обучении. Системы дистанционного обучения, включая Moodle [1], предоставляют преподавателям инструменты для создания различных типов тестовых заданий, позволяющих оценить знания обучающихся по заданной теме, модулю или курсу.

Классификация тестовых заданий осуществляется по ряду параметров, включая форму представления, тип ответов, уровень сложности и цели тестирования.

1. По типу ответа:

- С выбором одного правильного ответа — обучающемуся необходимо выбрать один верный вариант из предложенных (один из наиболее распространённых типов).
- С выбором нескольких правильных ответов — допускается выбор нескольких корректных вариантов (используется для оценки комплексного понимания).
- С кратким ответом — ответ даётся в виде слова, числа или короткой фразы (например, «Какой год основания МГУ?» → «1755»).
- С развернутым ответом (эссе) — открытая форма, требующая детального ответа, анализируется преподавателем вручную.
- Установление соответствия — необходимо сопоставить элементы из двух списков (например, термины и определения).
- Упорядочивание — задание на расстановку элементов в правильной последовательности (например, этапы алгоритма).
- Верно/неверно — бинарный тип, часто используемый для быстрого контроля понимания.

2. По цели тестирования:

- Диагностические — направлены на определение уровня знаний до начала обучения.

- Формирующие (промежуточные) — помогают отслеживать прогресс в процессе изучения материала.
- Контрольные (итоговые) — используются для итоговой аттестации, экзаменов, зачётов.

3. По уровню сложности:

- Базовые (низкого уровня) — проверяют знание фактов, терминов, определений.
- Среднего уровня — оценивают понимание, интерпретацию и применение знаний.
- Высокого уровня — требуют анализа, синтеза информации, критического мышления.

4. По способу генерации:

- Созданные вручную — преподаватель формирует задание и вводит вручную все параметры.
- Автоматически сгенерированные — задания создаются программно, по заданным параметрам, на основе текста, базы знаний или ИИ.

1.3. ПОДХОДЫ К АВТОМАТИЧЕСКОЙ ГЕНЕРАЦИИ ТЕСТОВ

Автоматическая генерация тестовых заданий — это процесс создания контрольных вопросов и ответов с использованием алгоритмических или интеллектуальных методов, что позволяет существенно сократить трудозатраты преподавателей и повысить масштабируемость образовательных процессов, особенно в условиях массового дистанционного обучения. Существуют различные подходы к генерации тестов, которые можно условно классифицировать следующим образом:

1. Алгоритмические (шаблонные) методы

Этот подход предполагает использование заранее заданных шаблонов, по которым с помощью подстановки переменных и данных автоматически формируются вопросы. Часто применяется в точных науках, где возможно варьировать числовые параметры и получать уникальные задания одного типа.

Преимущества:

- Простота реализации;
- Возможность массового создания однотипных заданий;
- Высокая предсказуемость структуры вопроса.

Недостатки:

- Ограниченная гибкость;
- Подходит преимущественно для формальных знаний;
- Трудности с генерацией заданий на понимание, анализ и синтез.

2. Основанные на базе знаний

В этом подходе используется структурированная база знаний по предметной области (например, онтология, база фактов), на основе которой формируются вопросы. Система «знает», какие понятия связаны, и может генерировать задания на основе логических связей между ними.

Преимущества:

- Высокая релевантность заданий;
- Возможность использования в обучающих экспертных системах.

Недостатки:

- Требуется трудозатрат на создание и поддержание базы знаний;
- Ограничена строгостью и полнотой базы.

3. Методы обработки естественного языка (NLP)

Здесь система анализирует текстовые материалы (например, лекции, статьи, учебники) и на их основе формирует вопросы. Такие системы используют алгоритмы выделения ключевых фраз, семантического анализа, определения логической структуры текста.

Типовые техники:

- Извлечение ключевых терминов и фактов;
- Переформулированные предложения в виде вопросов;
- Генерация отвлекающих вариантов ответов на основе синонимов или логических ошибок.

Преимущества:

- Возможность работы с неструктурированными текстами;
- Высокая адаптивность.

Недостатки:

- Требуется сложных NLP-алгоритмов;
- Вероятность генерации некорректных или неоднозначных вопросов.

4. Генерация на основе искусственного интеллекта

С внедрением современных языковых моделей (LLM — Large Language Models), таких как GigaChat, ChatGPT, GPT-4 и других, становится возможной генерация тестов, максимально приближенных по стилю и качеству к ручной работе преподавателя. Такие модели обучены на огромных объемах текстов и способны интерпретировать, переформулировать и генерировать задания по любой теме.

Преимущества:

- Универсальность и гибкость;
- Поддержка разных стилей, уровней сложности и языков;
- Возможность работы с произвольным текстом (например, загруженным файлом или пользовательским промптом).

Недостатки:

- Требуется вычислительные ресурсы;
- Необходим контроль качества (модели могут ошибаться);
- Зависимость от стороннего API или локального развертывания ИИ.

В рамках данной выпускной квалификационной работы реализуется гибридный подход, основанный на применении инструментов искусственного интеллекта для анализа текстовой информации и генерации тестов, с последующим автоматическим формированием XML-файла, совместимого с Moodle. Такой подход позволяет объединить адаптивность ИИ с формальной структурой LMS и повысить эффективность процесса тестирования.

1.4. РОЛЬ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА В ОБРАЗОВАТЕЛЬНЫХ ТЕХНОЛОГИЯХ

Искусственный интеллект (ИИ) [2] активно внедряется во все сферы человеческой деятельности, и образование не является исключением. Благодаря способности анализировать большие объемы данных, обучаться на основе паттернов и адаптироваться под пользователя, ИИ становится мощным инструментом трансформации традиционного обучения в персонализированную, динамичную и эффективную систему.

Основные направления применения ИИ в образовании:

1. Интеллектуальные обучающие системы (ITS)

Такие системы адаптируют подачу материала под уровень знаний обучающегося, его темп усвоения, пробелы в понимании. ITS применяют ИИ для:

- автоматического подбора задач;
- корректировки траектории обучения;
- выдачи индивидуальных рекомендаций.

2. Персонализация образовательного процесса

ИИ позволяет создавать уникальные сценарии взаимодействия с каждым студентом: подбирать задания, оценивать сильные и слабые стороны, предсказывать успеваемость. Это особенно ценно в условиях массового онлайн-обучения, когда индивидуальный подход становится затруднительным для преподавателя.

3. Автоматизированная проверка знаний

ИИ может анализировать не только тесты с выбором ответа, но и:

- открытые ответы;
- эссе;
- программный код;
- устную речь (с применением распознавания и анализа текста).

4. Генерация образовательного контента

Одним из наиболее перспективных применений является автоматическая генерация учебных заданий, что является ключевым аспектом данной работы. Современные языковые модели (LLM) способны:

- формулировать понятные и логичные вопросы;
- генерировать отвлекающие варианты ответов;
- адаптировать стиль под возраст, уровень сложности и тематику курса.

5. Поддержка преподавателя

ИИ может выступать в роли ассистента, предлагая:

- варианты тестов;
- планы занятий;
- материалы для объяснения сложных тем;
- автоматическое заполнение дневников и ведомостей.

Преимущества внедрения ИИ в образование:

- Снижение нагрузки на преподавателя;
- Повышение эффективности и точности оценки знаний;
- Возможность масштабирования обучения без потери качества;
- Обеспечение непрерывного, адаптивного образования.

Риски и ограничения:

- Необходимость контроля качества генерируемого контента;
- Этические вопросы (например, использование ИИ для списывания);
- Зависимость от сторонних сервисов (API, облачные платформы);
- Вопросы конфиденциальности и защиты персональных данных.

Таким образом, роль ИИ в образовательных технологиях заключается не только в автоматизации рутинных процессов, но и в создании качественно новой модели обучения, ориентированной на потребности и возможности каждого обучающегося. Применение ИИ, как в разрабатываемом плагине для Moodle, позволяет вывести процесс создания тестов на новый уровень, делая его более быстрым, точным и адаптивным.

1.5. ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ ПО ГЕНЕРАЦИИ ТЕСТОВ С ИСПОЛЬЗОВАНИЕ ИИ

С ростом популярности языковых моделей (LLM) на базе искусственного интеллекта, все чаще возникает потребность в их применении в образовательной сфере, в частности — для автоматической генерации тестов. На территории РФ особую актуальность приобретают отечественные ИИ-решения, способные работать с русскоязычным контентом, соблюдать требования по безопасности и быть интегрированы в локальные образовательные системы.

Сравнительный анализ отечественных ИИ-моделей

В рамках данной выпускной квалификационной работы был проведён сравнительный анализ трёх ведущих отечественных ИИ-платформ: GigaChat, YandexGPT и SberAI (RuGPT). Оценка производилась по следующим критериям (таб. 1):

Таблица 1. Сравнение отечественных ИИ-моделей

Критерий	GigaChat	YandexGPT	SberAI (RuGPT)
Поддержка русского языка	Отличная (нативная модель на русском языке)	Отличная	Средняя
Качество генерации тестов	Высокое, хорошо структурированные ответы	Среднее, часто уходит в «эссе»	Переменное, требуется доработка в формулировках
Интеграция через API	Простая, есть подробная документация	Ограниченная, требует регистрации в облаке	Ограниченная, нестабильный API
Контекстность	Учитывает длинные промты, адекватно работает	Есть склонность терять контекст	Зависит от промта

	с большими текстами		
Документация и доступ	Подробная документация, быстрый отклик	Закрытая часть API, требуется Yandex Cloud	Ограниченная (устаревшие описания)

На основании анализа можно сделать следующие выводы:

- GigaChat [4] от Сбера является наиболее подходящим вариантом для целей проекта:
 - Поддерживает работу с русскоязычными материалами на высоком уровне.
 - Обладает интуитивным API и легко подключается к веб-приложению.
 - Хорошо обрабатывает пользовательские промты, адаптируя формат под требования (в том числе для генерации вопросов с вариантами ответов).
 - Поддерживается и развивается, что важно для долгосрочной эксплуатации.
- YandexGPT [5] — перспективная модель, но плохо задокументированный доступ к API и ограничение по количеству запросов затрудняют её использование в образовательной инфраструктуре.
- SberAI (RuGPT) [6] — вариант с открытым исходным кодом, но требует дообучения и сложной настройки, что делает его малоприменимым для быстрой интеграции в Moodle.

Выбор в пользу GigaChat обоснован как техническими характеристиками, так и практической пригодностью для разработки образовательного плагина. Использование именно этой ИИ-модели в рамках проекта позволяет достичь баланса между качеством генерации, простотой интеграции и соответствием требованиям русскоязычной образовательной среды.

2. ПОСТАНОВКА ЗАДАЧИ И ПРОЕКТИРОВАНИЯ ПЛАГИНА ДЛЯ MOODLE

2.1. АНАЛИЗ ТРЕБОВАНИЙ К ПЛАГИНУ

Разработка плагина для Moodle требует чёткого понимания его функциональных и нефункциональных требований. Ниже представлены ключевые характеристики, которыми должен обладать плагин, а также обоснование этих требований с учётом целей и условий использования.

Функциональные требования:

1. Ввод исходных данных:

- Поддержка ручного ввода промта (в виде текстовой инструкции).
- Возможность загрузки текстового файла с учебным материалом.
- Объединение текста из файла и ручного ввода в единый запрос к ИИ.

2. Интеграция с ИИ-сервисом:

- Отправка запроса к API GigaChat.
- Обработка ответа в виде сгенерированных тестовых вопросов.

3. Генерация тестов:

- Формирование тестовых заданий в формате: вопрос + 4 варианта ответа, из которых один — правильный.
- Отображение результата на веб-интерфейсе.
- Сохранение теста в формате XML (структурированном под импорт в Moodle).

4. Скачивание и экспорт:

- Предоставление пользователю ссылки на скачивание XML-файла.
- Готовность файла к импорту через стандартный интерфейс Moodle.

5. Интерфейс пользователя:

- Простая и интуитивно понятная форма с текстовым полем, загрузкой файла и кнопкой генерации.
- Область вывода сгенерированного результата.
- Поддержка русскоязычного интерфейса.

Нефункциональные требования:

1. Безопасность:

- Проверка на наличие токена доступа к API.
- Ограничение загрузки файлов только форматом .txt.
- Исключение XSS-инъекций при выводе результата в браузер.

2. Надёжность:

- Обработка ошибок API (недоступность, неверный токен, неправильный формат ответа).
- Проверка наличия и корректности промта перед отправкой запроса.

3. Совместимость:

- Адаптация формата XML под текущую версию Moodle (используемую в АГУ).
- Поддержка популярных браузеров (Chrome, Firefox, Edge).

4. Производительность:

- Обработка запроса и генерация тестов не должна занимать более 10 секунд при типичных объёмах текста.

Проведённый анализ функциональных и нефункциональных требований к разрабатываемому плагину позволяет сформулировать полное техническое задание, соответствующее задачам проекта и реальным условиям его использования в учебном процессе. Ключевыми приоритетами являются:

- Юзабилити (удобство использования): интерфейс плагина должен быть интуитивно понятным даже для пользователей без технической подготовки. Возможность выбора между ручным вводом промта и загрузкой файла делает решение универсальным для преподавателей разных дисциплин.
- Интеграция с ИИ: выбор модели GigaChat продиктован её высокой производительностью при работе с русскоязычными текстами, что критически важно для преподавателей отечественных вузов. Интеграция через API обеспечивает автоматизацию генерации заданий без необходимости глубокого погружения в механику работы ИИ.
- Соответствие стандартам Moodle: сохраняемый XML-файл должен соответствовать внутреннему формату LMS Moodle, чтобы обеспечить его

беспроблемный импорт и дальнейшую работу с тестами внутри курса. Это гарантирует совместимость с системами, уже развернутыми в образовательных учреждениях, в частности, с установленной системой Moodle в АГУ.

- Надёжность и безопасность: реализация проверок входных данных и корректная обработка ошибок API обеспечивают устойчивость плагина в условиях реальной эксплуатации. Это особенно важно при внедрении в рабочую среду, где от стабильности системы зависит учебный процесс.
- Гибкость и масштабируемость: благодаря использованию API и модульной архитектуры, плагин может быть расширен в будущем, например, добавлением генерации заданий разных типов (на соответствие, с множественным выбором и т.п.) или поддержкой других языков.

Таким образом, плагин нацелен не только на автоматизацию рутинных задач, но и на повышение качества дистанционного образования, расширяя возможности преподавателей в подготовке материалов и адаптации тестов под конкретные курсы и аудитории.

2.2. ВЫБОР МЕТОДОВ ИИ ДЛЯ ГЕНЕРАЦИИ ТЕСТОВЫХ ЗАДАНИЙ

Процесс генерации тестовых заданий на основе учебного материала требует применения инструментов, способных эффективно обрабатывать естественный язык, выделять ключевые понятия и преобразовывать их в структуру вопросов с вариантами ответов. Одним из самых эффективных современных подходов к решению подобных задач является использование генеративных языковых моделей (LLM — Large Language Models).

В рамках разработки плагина для Moodle было решено применить метод текстовой генерации с использованием API доступа к LLM, на основе технологии промт-инжиниринга. Суть подхода заключается в том, что пользователю (преподавателю) предлагается ввести текстовый запрос (промт) вручную или загрузить текстовый файл с учебным материалом. Полученный текст обрабатывается и направляется в языковую модель, которая генерирует тестовые вопросы и варианты ответов в соответствии с заранее заданным шаблоном.

Такой подход был выбран по следующим причинам:

- Гибкость и масштабируемость: модель способна работать с текстами различной тематики и объема, автоматически подстраиваясь под стиль и терминологию;
- Минимизация усилий со стороны преподавателя: отсутствует необходимость вручную создавать вопросы — модель формирует их автоматически;
- Универсальность интеграции: промт-инжиниринг легко адаптируется под разные цели без необходимости дополнительного обучения модели.

На этапе проектирования рассматривались различные отечественные языковые модели, включая GigaChat, SberAI (RuGPT) и YandexGPT. Подробное сравнительное описание приведено в пункте 1.5. С практической и технической точек зрения наилучшим выбором стал GigaChat от Сбера, поскольку:

- Поддерживает API-доступ с прозрачной процедурой авторизации через OAuth2 и передачей запросов в JSON-формате;
- Обладает высокой точностью при генерации тестов на русском языке, включая тематические академические тексты;

- Демонстрирует гибкое поведение при изменении формулировок промта, что критически важно при работе с различными дисциплинами;
- Имеет стабильную и предсказуемую структуру ответов, что упрощает последующий парсинг [7] и автоматическую генерацию XML-файла для Moodle.

Особое внимание было уделено формату взаимодействия с моделью. Для обеспечения корректности выходного текста и его автоматической обработки на стороне плагина используется строго заданный шаблон промта. Он включает указание количества вопросов, формат с буквенными вариантами ответов (a, b, c, d) и финальной пометкой правильного ответа. Такой формат позволяет в дальнейшем автоматически распознать структуру и преобразовать её в XML-структуру, пригодную для загрузки в Moodle.

Также важно отметить, что выбранная модель позволяет масштабировать процесс — при необходимости преподаватель может загружать объемные текстовые лекции, по которым система будет формировать тесты автоматически, без потери качества. Благодаря этому достигается высокий уровень автоматизации и индивидуализации образовательного процесса.

Таким образом, выбор метода генерации и конкретной языковой модели был обусловлен не только качественными характеристиками ИИ, но и совместимостью с архитектурой Moodle, удобством API-интеграции, безопасностью аутентификации и универсальностью обработки текстовых данных.

2.3. ВЫБОР ТЕХНОЛОГИЙ РАЗРАБОТКИ

Проектирование и реализация плагина для Moodle требует осознанного выбора технологического стека. Важнейшими критериями стали:

- совместимость с платформой Moodle;
- простота интеграции с API искусственного интеллекта (в частности, GigaChat);
- безопасность и масштабируемость;
- доступность для модификации и поддержки в будущем;

1. Язык серверной части: PHP

PHP [8] — это серверный скриптовый язык программирования общего назначения, специально разработанный для веб-разработки. Его основное назначение заключается в создании динамических веб-сайтов и приложений, позволяя разработчикам взаимодействовать с базами данных, обрабатывать формы, создавать сессионные данные пользователей и автоматизировать многие процессы на стороне сервера.

Moodle является системой, написанной на языке PHP, и вся логика его расширений строится вокруг встроенных API и правил, заложенных в эту экосистему. Поэтому выбор PHP как основного языка разработки плагина — это решение, полностью соответствующее следующим требованиям:

- Полная совместимость: использование PHP обеспечивает бесшовную интеграцию с ядром Moodle, включая работу с его внутренними объектами, пользователями, доступами и формами.
- Скорость разработки: PHP позволяет быстро разрабатывать прототипы и рабочие решения без необходимости настраивать внешние серверы.
- Минимальные зависимости: решение полностью самодостаточно и не требует установки сторонних интерпретаторов или сред выполнения.

Альтернатива: Python

Python [10] активно используется при работе с AI/ML и обладает удобными библиотеками (requests, fastapi и др.). Однако его использование потребовало бы:

- развертывания дополнительного микросервиса вне Moodle;
- настройки взаимодействия через REST API между Moodle и Python-сервером;

- управления безопасностью, CORS и авторизацией отдельно.

Поэтому был сделан выбор в пользу PHP как более стабильной и интегрированной среды.

2. Взаимодействие с API GigaChat: cURL и JSON

Для отправки HTTP-запросов и получения данных от GigaChat API используется встроенная в PHP библиотека cURL. Она позволяет:

- отправлять POST-запросы с необходимыми заголовками (авторизация, content-type и т.д.);
- обрабатывать ответы, включая HTTP-коды и сообщения об ошибках;
- эффективно работать с JSON-форматом.

Полученные ответы декодируются с помощью `json_decode` и далее анализируются и обрабатываются в рамках PHP-логики.

3. Формат представления данных: XML [11]

Результатом работы плагина является сформированный XML-файл с тестовыми вопросами. Moodle поддерживает импорт тестов именно в этом формате. Структура файла формируется динамически с помощью встроенного класса SimpleXMLElement.

- Каждому вопросу в XML соответствует тег `<question>`, содержащий текст вопроса и вложенные варианты ответов.
- Правильный ответ обозначается через атрибут `correct="true"`.

Это делает плагин совместимым с Moodle без необходимости дополнительных преобразований.

4. Клиентская часть: HTML + JavaScript + CSS

HTML (Hypertext Markup Language) [13] представляет собой язык разметки, применяемый для организации и визуализации веб-страниц и их содержимого.

JavaScript [14] – это программный язык, главным образом используемый для разработки веб-сайтов. Он позволяет сделать веб-страницы более динамичными, добавляя такие элементы, как всплывающие уведомления, анимированные эффекты.

CSS [15], или каскадные таблицы стилей, представляет собой язык, используемый для визуального оформления веб-страниц. С его помощью задаются правила отображения HTML-элементов, определяющие их внешний вид, включая цветовую

гамму, шрифтовое оформление, расположение на странице и другие параметры, отвечающие за визуальную составляющую.

Пользователь взаимодействует с системой через простую HTML-страницу, содержащую форму для:

- ввода текста вручную;
- загрузки текстового файла;
- отправки промта на сервер без перезагрузки страницы.

Для оформления интерфейса используется CSS-стили, определенные в отдельном файле, обеспечивающие адаптивную и визуально удобную структуру страницы.

JavaScript используется для:

- валидации данных формы;
- динамического отображения статуса (загрузка, ответ, ошибка);
- отображения ссылки на скачивание XML-файла.

Таким образом, обеспечивается современное и удобное взаимодействие с пользователем без лишних перезагрузок и ожиданий.

5. Безопасность и изоляция логики

Плагин реализован как изолированный модуль:

- все действия по получению токена, отправке запроса и генерации тестов проходят в одном серверном скрипте (generate.php);
- логика четко разделена: обработка входных данных, работа с API, генерация XML.

В таблице 2 представлена обобщённая структура применяемых технологий, их назначение и краткое обоснование выбора. Это позволяет наглядно продемонстрировать, как каждый компонент плагина опирается на определённые инструменты, согласованные между собой в рамках общей архитектуры.

Таблица 2. Обобщенная структура используемых технологий

Компонент	Технологии	Обоснование выбора
Серверная часть	PHP	Совместимость с Moodle, простота, встроенные API
Взаимодействие с API	PHP + cURL	Гибкость, управление заголовками, устойчивость
Формат тестов	XML	Стандартизованный формат импорта в Moodle
Конфигурация	config.php	Безопасность, модульность
Клиентская часть	HTML + JS + CSS	Интерактивность интерфейса

Таким образом, выбранный технологический стек обеспечивает надежную и удобную реализацию плагина, полностью совместимую с экосистемой Moodle. Использование PHP как основного языка, работа с API через cURL [17], оформление интерфейса средствами HTML, CSS и JavaScript, а также формирование выходных данных в формате XML позволяет достичь поставленных целей при минимальных издержках и высокой гибкости решения.

2.4. АРХИТЕКТУРА И СТРУКТУРА РАЗРАБАТЫВАЕМОГО ПЛАГИНА

Проектируемый плагин представляет собой автономный веб-интерфейс, интегрируемый в экосистему Moodle и предназначенный для генерации тестовых заданий с использованием искусственного интеллекта. Архитектура решения построена по клиент-серверной модели, где клиентская часть обеспечивает взаимодействие с пользователем, а серверная — обработку данных, обращение к ИИ-модели и формирование выходного файла.

На рисунке 1 представлена общая архитектура разрабатываемого плагина для генерации тестов на основе искусственного интеллекта. Схема демонстрирует последовательность взаимодействия между пользователем, клиентской и серверной частями плагина, внешним API GigaChat и формируемым XML-файлом, пригодным для импорта в систему Moodle.

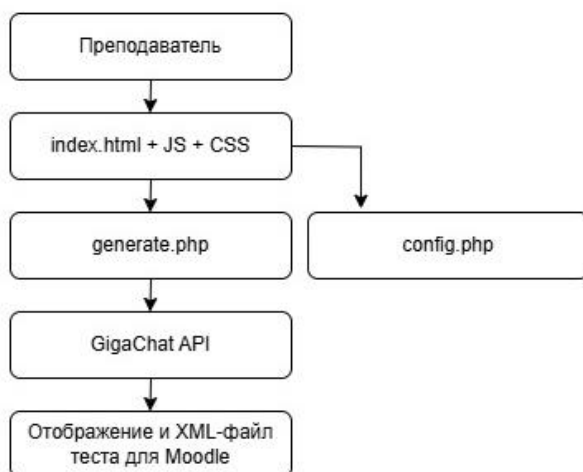


Рисунок 1. Архитектура плагина

Описание компонентов архитектуры:

1. Пользовательский интерфейс (frontend)

Представлен HTML-страницей с подключёнными стилями (CSS) и логикой на JavaScript. Позволяет:

- ввести текст вручную;
- загрузить текстовый файл;
- отправить промт на сервер;

- отследить статус генерации;
- получить файл на скачивание XML-файла.

2. Обработка запроса (generate.php)

Серверный PHP-скрипт, выполняющий следующие функции:

- загрузка конфигурации (config.php);
- авторизация через API GigaChat (получение токена);
- формирование запроса с промптом;
- отправка запроса к GigaChat;
- получение и обработка ответа;
- извлечение сгенерированных вопросов;
- построение XML-файла и сохранение его на сервере;
- отображение результата пользователю.

3. Файл конфигурации (config.php)

Изолирует чувствительные данные (client_id, client_secret, scope), позволяя обеспечить безопасность и модульность.

4. Внешнее API GigaChat (Сбер)

Обеспечивает генерацию тестовых заданий по заданному тексту. Взаимодействие осуществляется через REST API, в формате JSON.

5. XML-файл (generated_test.xml)

Формируется сервером на основе ответа от GigaChat и соответствует структуре, пригодной для импорта в Moodle. Каждое задание включает текст вопроса, варианты ответа и указание верного.

Разработанная архитектура обеспечивает модульность, удобство сопровождения, а также гибкость при дальнейшей доработке и расширении функциональности плагина в рамках платформы Moodle.

2.5. ПРОЕКТИРОВАНИЕ ИНТЕРФЕЙСА ПОЛЬЗОВАТЕЛЯ

Интерфейс пользователя (UI) [18] плагина играет ключевую роль в его удобстве и доступности для преподавателей и других пользователей Moodle, не обладающих специальными техническими знаниями. Цель проектирования UI — создать простую, интуитивно понятную форму, позволяющую за считанные шаги сгенерировать тест на основе текста, с минимальным количеством действий со стороны пользователя.

Разрабатываемый интерфейс включает следующие ключевые элементы:

- Поле для ввода текста вручную — основная зона, куда преподаватель может вставить учебный материал, на основе которого будет сформирован тест.
- Форма загрузки файла — альтернативный способ подачи текста. Позволяет загрузить .txt-файл с содержанием, по которому будет сгенерирован тест.
- Кнопка отправки данных — после ввода текста или загрузки файла пользователь может отправить запрос на генерацию теста. Кнопка инициирует AJAX-запрос без перезагрузки страницы.
- Область отображения статуса — информирует пользователя о текущем процессе: получении токена, передаче данных, ожидании ответа, успешной генерации или возникшей ошибке.
- Ссылка на скачивание XML-файла — появляется только после успешной генерации теста. Файл полностью совместим с системой Moodle и может быть сразу импортирован в раздел «Вопросы» или «Тесты».

Используемые технологии

- HTML5 — для построения структуры страницы и формы;
- CSS — для стилизации и визуальной организации компонентов: отступов, шрифтов, цветовых акцентов, читаемости;
- JavaScript (в чистом виде) — для динамического управления поведением интерфейса: отправка данных, загрузка статуса, отображение результатов.

Особенности дизайна

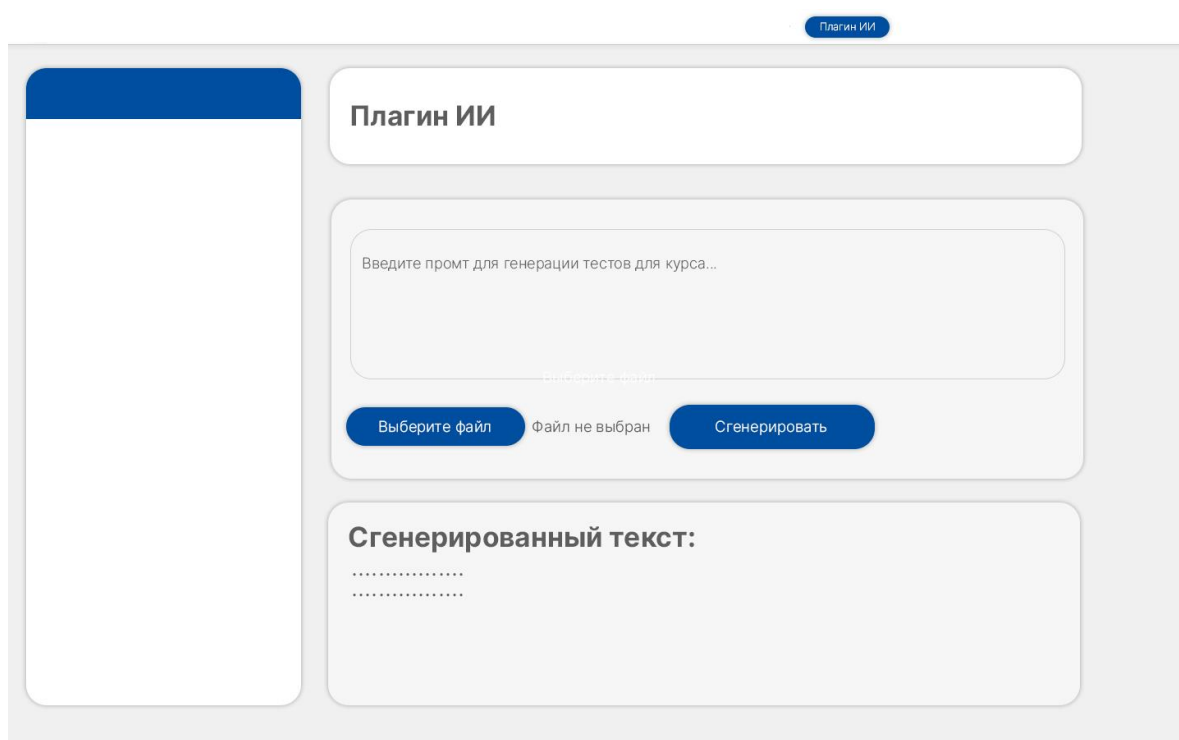
Интерфейс выполнен в минималистичном стиле, соответствующем общей эстетике Moodle. Основной акцент сделан на функциональность, понятность и отзывчивость (отсутствие полной перезагрузки страницы, мгновенная обратная связь).

Расширяемость

В случае необходимости, интерфейс может быть дополнен следующими элементами:

- выпадающим списком для выбора количества вопросов;
- опциями форматирования вывода;
- выбором модели ИИ при наличии нескольких доступных.

На рисунке 2 представлена пользовательская форма плагина, разработанная для взаимодействия с системой генерации тестов на основе ИИ. Интерфейс демонстрирует простую и интуитивную структуру, обеспечивающую ввод текста, загрузку файла, отправку запроса и получение итогового результата в формате XML.



The screenshot shows a Moodle plugin interface titled "Плагин ИИ". It features a large text input field with the placeholder "Введите промт для генерации тестов для курса...". Below the input field are two buttons: "Выберите файл" (which shows "Файл не выбран") and "Сгенерировать". At the bottom, there is a section labeled "Сгенерированный текст:" followed by a large text area containing several lines of dots, representing the generated output.

Рисунок 2. Внешний вид интерфейса плагина для генерации тестов в Moodle

2.6. ОБЕСПЕЧЕНИЕ БЕЗОПАСНОСТИ И ЗАЩИТЫ ДАННЫХ

При разработке плагина особое внимание уделяется вопросам безопасности, поскольку взаимодействие с внешним API и обработка пользовательских данных требуют строгого соблюдения стандартов информационной безопасности. Основные меры, реализованные в рамках проекта:

1. Изоляция конфиденциальных данных

Все конфиденциальные данные (например, `client_id` и `client_secret` для авторизации в GigaChat) хранятся в отдельном файле конфигурации: `config.php` который недоступен напрямую из браузера. Это снижает риск компрометации ключей доступа и токенов.

2. Проверка достоверности входных данных

На стороне сервера входящие данные проверяются на доступность, корректность и безопасность:

- проверка на наличие и не пустоту пробелов;
- фильтрация загружаемых файлов;
- ограничение типов и размеров загружаемых данных.

3. Защита от XSS

При формировании XML-файла используется функция `htmlspecialchars`, предотвращающая введение потенциально опасного HTML или JS-кода. Все пользовательские данные проверяются перед сохранением или выводом.

4. Использование защищенных подключений

Взаимодействие с API GigaChat происходит исключительно по протоколу HTTPS с проверкой SSL-сертификата, что гарантирует защиту данных в процессе передачи и предотвращает атаки типа "Man-in-the-middle".

5. Ограничения доступа

Файлы плагина размещены в защищенном каталоге, доступном только авторизованным пользователям Moodle с соответствующими правами. Это исключает возможность непосредственного несанкционированного вызова скрипта.

Таким образом, в процессе проектирования и реализации плагина были приняты меры по предотвращению угроз безопасности, связанных как с пользовательскими

вводами, так и с операциями. Принятая модель обеспечивает надежную работу плагина без риска утечки или повреждения данных.

3. РАЗРАБОТКА ПЛАГИНА ДЛЯ MOODLE

3.1. РАЗРАБОТКА ПЛАГИНА

Разработка плагина велась поэтапно с чётким разделением логики на: инициализацию структуры, взаимодействие с API GigaChat, обработку пользовательских данных и генерацию тестов в формате, совместимом с Moodle. Работа над серверной и клиентской частями велась параллельно, с учётом требований к безопасности, масштабируемости и удобству использования. Далее рассмотрены ключевые этапы разработки.

1. Создание структуры плагина

Плагин представляет собой автономный модуль, состоящий из следующих компонентов:

- index.html — основная страница интерфейса пользователя, содержащая форму для ввода промта, загрузки файла и отображения результата;
- generate.php — серверный скрипт, осуществляющий авторизацию в GigaChat API, генерацию тестов, их парсинг, отображение и сохранение в формате XML;
- script.js — JavaScript-файл, обрабатывающий отправку формы, запросы к серверу и отображение ответа без перезагрузки страницы;
- styles.css — стили оформления пользовательского интерфейса;
- config.php — конфигурационный файл с параметрами доступа к GigaChat API.

На рисунке 3 представлена структура файлов, входящих в состав разработанного плагина. Каждый элемент выполняет отдельную функцию.

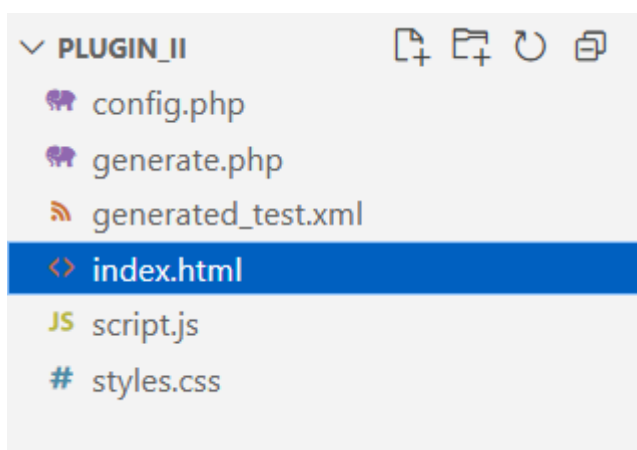


Рисунок 3. Структура файлов плагина в корневом каталоге

Создание конфигурационного файла `config.php` (рис. 4) позволяет изолировать чувствительные данные, такие как `client_id`, `client_secret` и `scope`, упрощая их замену или адаптацию при миграции плагина:

```
config.php
1  <?php
2
3  return [
4      'client_id' => 'd3176f64-62b9-4e3e-8af1-9440eb7445d5',
5      'client_secret' => '100a2b50-0f30-4e3e-8af1-9440eb7445d5',
6      'scope' => 'GIGACHAT_API_PERS',
7  ];
```

Рисунок 4. Конфигурация доступа к GigaChat API

2. Интеграция с ИИ GigaChat API

Ключевым элементом функциональности плагина является взаимодействие с GigaChat API — языковой моделью, разработанной Сбербанком, которая используется для генерации тестовых заданий на основе пользовательского запроса или текстового контекста.

Для подключения к API необходимо получить OAuth-токен. На рисунке 5 представлен интерфейс личного кабинета разработчика GigaChat, где можно сгенерировать необходимые параметры доступа — Client ID и Client Secret:

The screenshot shows a web interface titled "Сохраните ваш Authorization Key" (Save your Authorization Key). It explains that the Authorization Key is a base64-encoded pair of Client ID and Client Secret. Below this, there are three input fields: "Authorization Key" (containing a long base64 string), "Client Secret" (containing "100a2b50-0f30-4e3e-8af1-9440eb7445d5"), and "Client ID" (containing "d3176f64-62b9-4e3e-8af1-9440eb7445d5"). Each field has a copy icon. At the bottom, there is a blue "Готово" (Done) button. A warning message at the bottom states: "Не закрывайте это окно, пока не сохраните данные. Данные отображаются только один раз и не хранятся в системе. При компрометации или утере вы сможете получить новое значение." (Do not close this window until you save the data. Data is displayed only once and is not stored in the system. In case of compromise or loss, you will be able to get a new value.)

Рисунок 5. Интерфейс личного кабинета GigaChat с параметрами API-доступа

Процесс авторизации реализован в `generate.php` с использованием библиотеки `cURL`, встроенной в PHP. Скрипт отправляет POST-запрос на эндпоинт авторизации

GigaChat, получает токен и использует его в дальнейшем для отправки запроса на генерацию (рис. 6).

```
$ch = curl_init('https://ngw.devices.sberbank.ru:9443/api/v2/oauth');
curl_setopt_array($ch, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_POST => true,
    CURLOPT_HTTPHEADER => [
        "Authorization: Basic $auth_key",
        "RqUID: " . $client_secret,
        "Accept: application/json",
        "Content-Type: application/x-www-form-urlencoded"
    ],
    CURLOPT_POSTFIELDS => http_build_query(['scope' => $scope]),
    CURLOPT_SSL_VERIFYPEER => true,
    CURLOPT_SSL_VERIFYHOST => 2,
]);

$response = curl_exec($ch);
$data = json_decode($response, true);
$token = $data['access_token'] ?? null;
curl_close($ch);

if (!$token) {
    http_response_code(500);
    echo "Не удалось получить токен.";
    exit;
}
```

Рисунок 6. Авторизация и получение токена

В случае неуспешного получения токена выводится сообщение об ошибке, и выполнение скрипта прерывается. Это повышает устойчивость системы к сбоям.

3. Обработка пользовательских данных

Плагин поддерживает два источника пользовательского ввода (рис 7.): текстовую инструкцию (prompt) и загружаемый .txt-файл. Если файл был прикреплен, его содержимое объединяется с промптом для формирования общего контекста генерации.

```
$instruction = $_POST['prompt'] ?? '';
$context = '';

if (isset($_FILES['textfile']) && $_FILES['textfile']['tmp_name']) {
    $context = file_get_contents($_FILES['textfile']['tmp_name']);
}

if (!$instruction && !$context) {
    http_response_code(400);
    echo "Инструкция и файл отсутствуют";
    exit;
}

$prompt = $instruction;
if ($context) {
    $prompt .= "\n\nВот текст документа, на основе которого нужно составить вопросы:\n$context";
}
```

Рисунок 7. Объединение промта и содержимого загружаемого файла

4. Отправка запроса на генерацию теста

Формируется JSON-запрос в формате, поддерживаемом GigaChat API. Указываемая модель, структура сообщений и температурный параметр генерации (рис. 8).

```
$headers = [  
    "Authorization: Bearer $token",  
    "Content-Type: application/json",  
];  
  
$body = json_encode([  
    "model" => "GigaChat",  
    "messages" => [[  
        "role" => "user",  
        "content" => $prompt  
    ]],  
    "temperature" => 0.8,  
]);
```

Рисунок 8. Формирование запроса к GigaChat

После этого отправляется POST-запрос и результат сохраняется в переменную \$content (рис. 9).

```
$ch = curl_init('https://gigachat.devices.sberbank.ru/api/v1/chat/completions');  
curl_setopt_array($ch, [  
    CURLOPT_RETURNTRANSFER => true,  
    CURLOPT_POST => true,  
    CURLOPT_HTTPHEADER => $headers,  
    CURLOPT_POSTFIELDS => $body,  
    CURLOPT_SSL_VERIFYPEER => true,  
    CURLOPT_SSL_VERIFYHOST => 2,  
]);  
  
$response = curl_exec($ch);  
$data = json_decode($response, true);  
$content = $data['choices'][0]['message']['content'] ?? 'Не удалось получить ответ';  
curl_close($ch);
```

Рисунок 9. Выполнение запроса к GigaChat и получение ответа

5. Парсинг сгенерированного теста, и генерация XML

После получения ответа от GigaChat, необходимо извлечь структуру теста: текст вопросов, варианты ответов и корректные ответы. Для этого в generate.php используется регулярное выражение, способное обрабатывать различные стили оформления. Далее создаётся XML-объект SimpleXMLElement, в который добавляются распознанные элементы (рис. 10).

```

$xml = new SimpleXMLElement('<quiz></quiz>');

$pattern = '/
(?:Вопрос\s*(d+[:\s])?\s*)          # заголовок "Вопрос 1:"
(?:\s*\r\n)+                        # текст вопроса

(?:[-\s]*\s*)?a)\s*(.*)\s*\r\n)+    # вариант A
(?:[-\s]*\s*)?b)\s*(.*)\s*\r\n)+    # вариант B
(?:[-\s]*\s*)?c)\s*(.*)\s*\r\n)+    # вариант C
(?:[-\s]*\s*)?d)\s*(.*)\s*\r\n)+    # вариант D

(?:\s*)?Ответ[:\s])?(?:\s*)?\s*([a-d]) # жирный или обычный ответ
/isuUx';
$content = preg_replace('/\s*|###|^\s+/m', '', $content);

preg_match_all($pattern, $content, $matches, PREG_SET_ORDER);

if (!$matches) {
    echo "<p style='color:red;'>✗ Не удалось распознать структуру теста. Убедитесь, что каждый вопрос оформ
} else {
    foreach ($matches as $match) {
        $question = $xml->addChild('question');
        $question->addChild('text', htmlspecialchars(trim($match[1])));
        $options = ['a' => $match[2], 'b' => $match[3], 'c' => $match[4], 'd' => $match[5]];
        $correct_letter = strtolower($match[6]);

        foreach ($options as $letter => $text) {
            $option = $question->addChild('option', htmlspecialchars(trim($text)));
            $option->addAttribute('correct', $letter === $correct_letter ? 'true' : 'false');
        }

        $xml_file = 'generated_test.xml';
        $xml->asXML($xml_file);

        echo "<a href='\$xml_file' download> Скачать XML</a>";
    }
}

```

Рисунок 10. Парсинг текста и формирование структуры XML

XML-файл формируется в соответствии со структурой Moodle XML-формата, включающей теги `<question>`, `<name>`, `<questiontext>`, `<answer>` и атрибуты корректности. Это позволяет импортировать сгенерированные тесты напрямую в систему Moodle через встроенный механизм импорта.

6. Реализация клиентской части

Пользовательский интерфейс плагина выполнен в виде одностраничного HTML-приложения с асинхронной отправкой данных. Основной файл `index.html` содержит визуальные элементы: форму генерации, поле загрузки файла и блок для вывода результата.

Файл `script.js` содержит JavaScript-код, обрабатывающий отправку формы. Он собирает данные, отправляет их на сервер через `fetch()`, а затем отображает результат без перезагрузки страницы.

Для визуального оформления интерфейса используется файл `styles.css`. Он придаёт современный и адаптивный вид форме, заголовкам и блоку с результатом генерации. Пользователь получает простой и интуитивно понятный опыт взаимодействия с плагином.

Таким образом, разработанный плагин обеспечивает полный цикл генерации тестов: от получения исходного текста — до формирования совместимого с Moodle XML-файла. Благодаря асинхронной клиентской части, изолированной конфигурации и интеграции с GigaChat, плагин демонстрирует гибкость, масштабируемость и простоту использования.

3.2. ТЕСТИРОВАНИЕ ПЛАГИНА

Задача проверки созданного расширения заключалась в оценке его работоспособности, надежности и правильности взаимодействия с платформой Moodle. Испытания проводились в разнообразных условиях, с использованием различных наборов входных параметров и последующим изучением полученных результатов.

Начальным шагом тестирования плагина стала проверка его функциональности при создании вопросов на основе текстового запроса, введенного непосредственно в предназначенное для этого поле, без загрузки каких-либо файлов. Пользователь задает текстовую инструкцию, к примеру: «Сформируй тест по теме: «», 5 вопросов, с 4 вариантами ответа (один правильный). Формат: Вопрос 1:, Ответ:». Плагин, получив данную инструкцию, отправляет запрос в GigaChat, получает сгенерированный результат и отображает его в пользовательском интерфейсе (рис. 11).

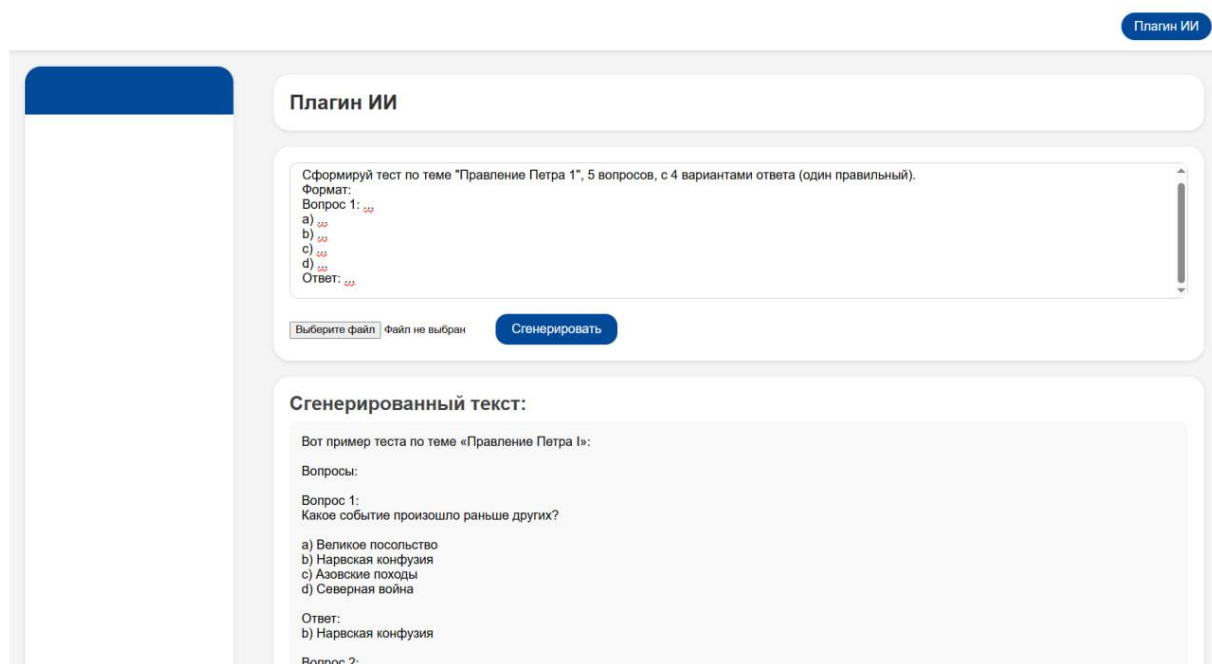


Рисунок 11. Генерация теста по пользовательскому промту

На изображении 12 показан результат сформированного набора вопросов и ответов, полученных от GigaChat, а также активная кнопка для скачивания XML-файла.

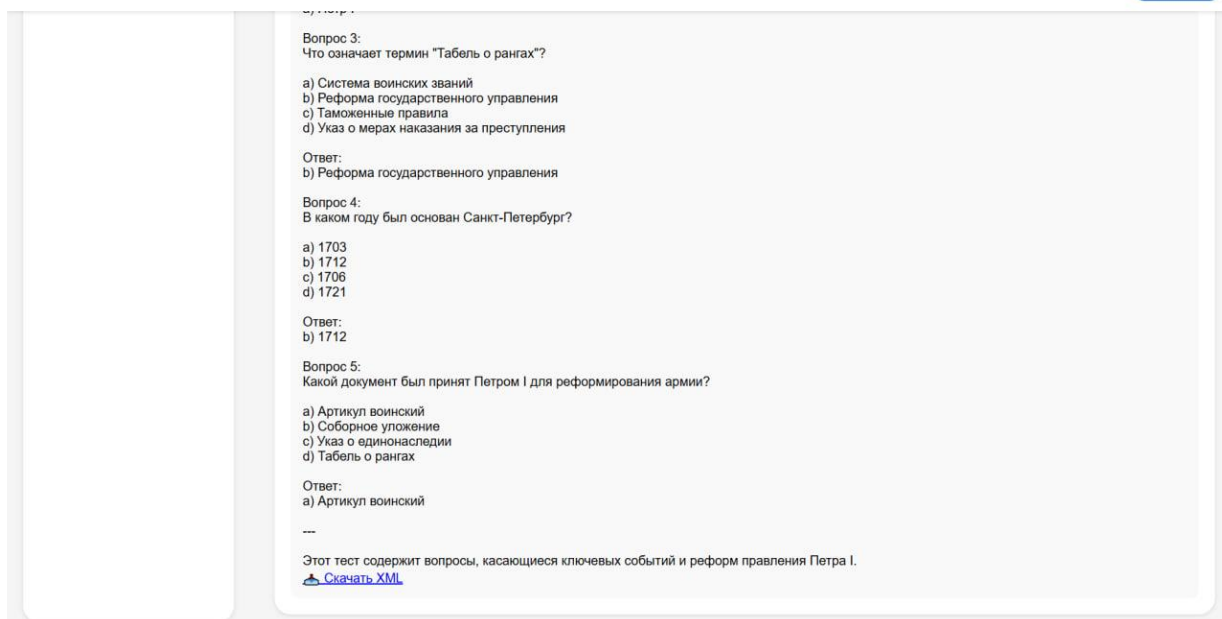


Рисунок 12. Результат генерации теста по промту с загрузкой XML-файла

На рисунке 13 представлено содержимое XML-файла, сформированного на основе полученного от GigaChat результата. Структура соответствует формату XML: каждый вопрос представлен как отдельный элемент `<question>`, с вложенными вариантами ответа и указанием правильного.

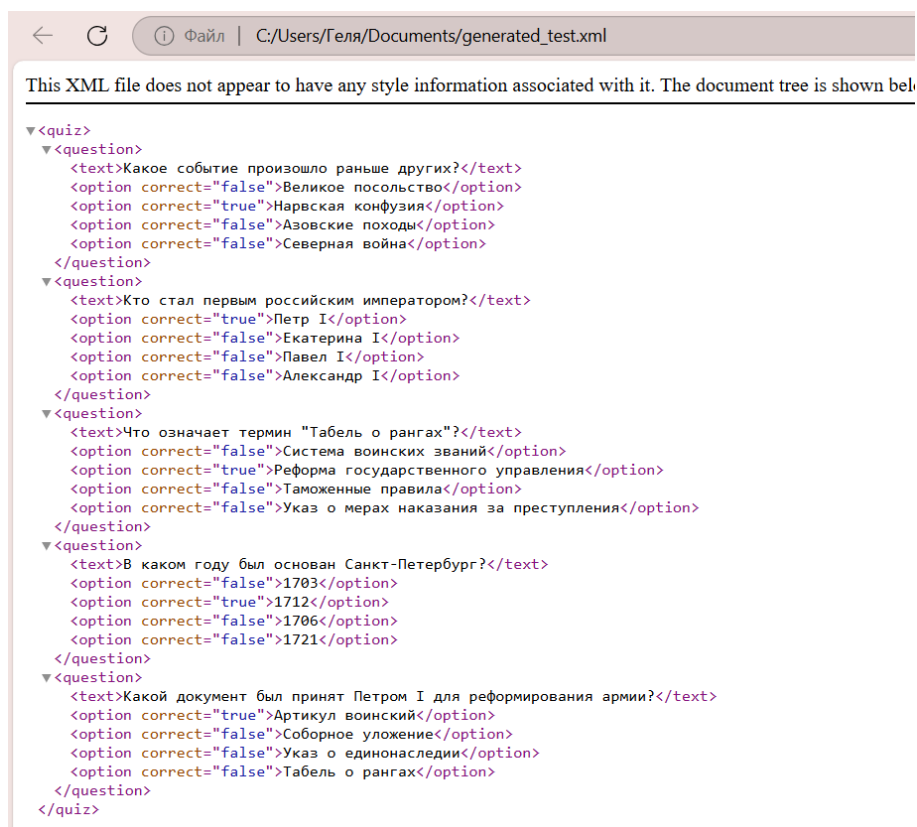


Рисунок 13. XML-файл, сгенерированный по результатам генерации теста по промту

Следующий режим тестирования включал загрузку текстового файла (рис. 14) в дополнение (или вместо) промта. Такой подход полезен при генерации тестов на основе лекционного материала, выдержек из книг и прочего контентного блока.

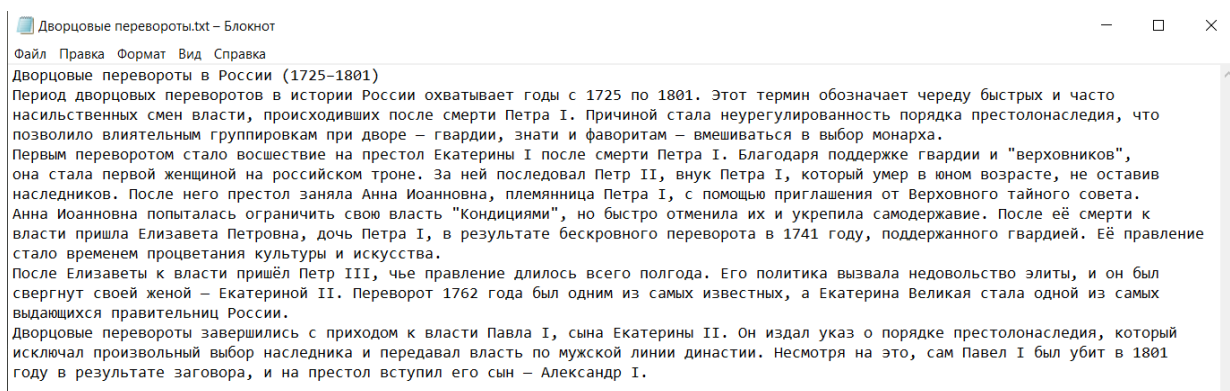


Рисунок 14. Пример содержимого загружаемого текстового файла

На рисунке 15 показан интерфейс плагина после выбора файла и (опционально) ввода вспомогательного текста в поле промта. Пользователь может передать объёмный контекст для генерации теста.

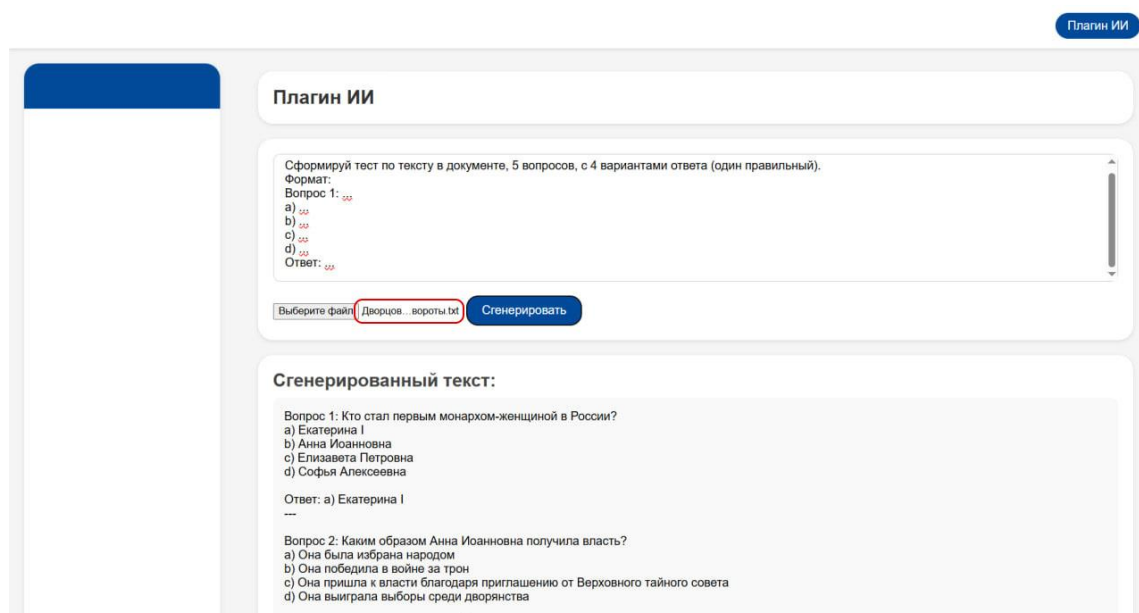


Рисунок 15. Интерфейс с выбранным .txt-файлом и дополнительным промтом

Результат генерации отображён на следующем изображении (рис. 16). В интерфейсе видны сгенерированные вопросы, варианты ответов, а также кнопка для скачивания XML-файла.

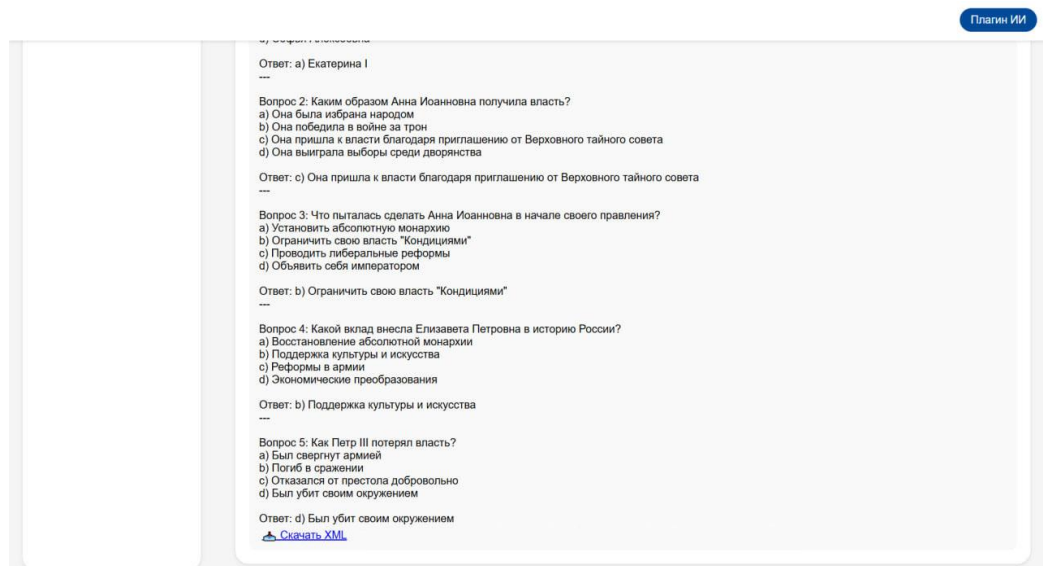


Рисунок 16. Отображение теста, сгенерированного на основе загружаемого файла

На рисунке 17 представлено содержимое XML-файла, сформированного на основе анализа текстового файла. Видно, что структура корректно создана: вопросы и ответы разбиты по тегам, предусмотрена логика импорта в Moodle.

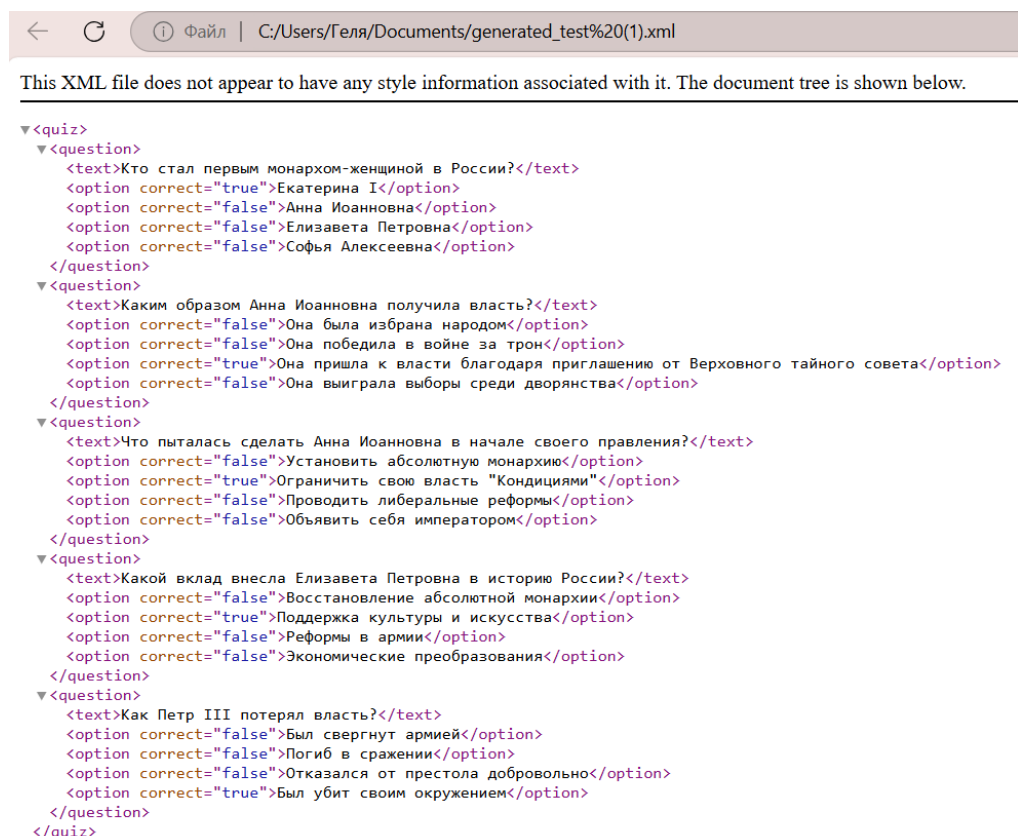


Рисунок 17. Сформированный XML-файл на основе текста из .txt-файла

Также в ходе тестирования была рассмотрена ситуация, при которой пользователь нажимает кнопку «Сгенерировать», не указав текстовую инструкцию и не

загрузив файл. На изображении 18 продемонстрирован интерфейс плагина в подобной ситуации: системы корректно выводит предупреждение – «Инструкция и файл отсутствуют», что предотвращают отправку пустого запроса и указывает пользователю на необходимость ввода исходных данных.

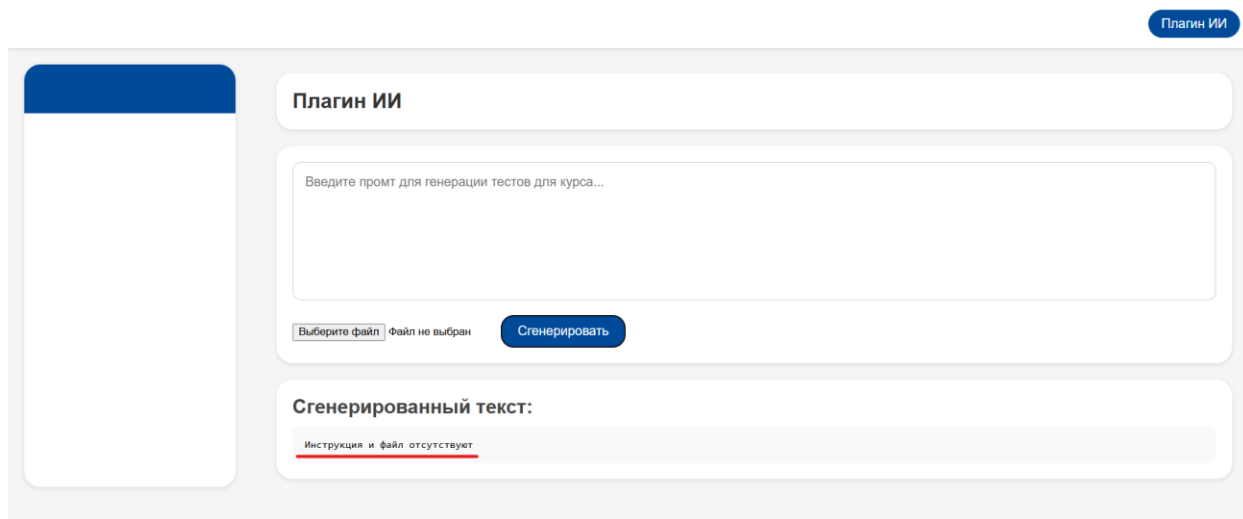


Рисунок 18. Сообщение об ошибке при отсутствии промта и загружаемого файла

Плагин успешно справился с поставленными задачами. На всех этапах были получены корректные ответы от GigaChat, данные обрабатывались и сохранялись в нужном формате. Благодаря модульной архитектуре и простой логике взаимодействия, плагин показал надёжность, универсальность и готовность к использованию в образовательной среде.

3.3. УСТРАНЕНИЕ ОШИБОК И ОПТИМИЗАЦИЯ РАБОТЫ

В процессе разработки и тестирования плагина возникли технические и логические ошибки, связанные как с серверной, так и с клиентской частью. На данном этапе выполнялось поэтапное устранение неполадок и оптимизация производительности и стабильности системы.

1. Ошибки авторизации и сертификаты

Одной из первых проблем стало получение OAuth-токена от GigaChat API. При отправке POST-запроса с помощью cURL на стороне сервера возникала ошибка SSL-соединения. Причиной стала необходимость явного подключения корневых сертификатов для работы через HTTPS. Для устранения этого был подключён файл доверенных сертификатов (cacert.pem).

2. Обработка ошибок API и логика прерывания

Была реализована централизованная логика обработки ошибок при авторизации и отправке запроса. При отсутствии токена, неверных параметрах или сбое соединения пользователь получал понятное сообщение в интерфейсе. Также добавлена проверка на наличие ключевых полей в ответе API перед выполнением дальнейших шагов.

3. Устойчивость парсинга тестов

Из-за различий в стиле генерации (маркеры 1., A), —, *) были улучшены регулярные выражения, применяемые при разборе текста. Также введены fallback-механизмы, позволяющие обрабатывать неполные или неоднозначные фрагменты — такие блоки помечаются, но не сбивают структуру всего XML-файла.

4. Повышение удобства и визуальных откликов

На клиентской стороне был реализован блок обработки ошибок и подсказок. Также добавлена индикация выполнения запроса (loader), автоочистка формы после отправки и вывод уведомлений в случае успеха или сбоя. Это повысило отзывчивость интерфейса и упростило повторное использование.

В результате проведённой отладки и оптимизации плагин стал надёжнее, устойчивее к сбоям и более дружелюбным к конечному пользователю. Были устранены технические ограничения, добавлены элементы обработки исключений и улучшена поддержка нестандартных сценариев работы.

ЗАКЛЮЧЕНИЕ

Выпускная квалификационная работа была посвящена разработке программного решения — плагина для системы дистанционного обучения Moodle, использующего возможности искусственного интеллекта (GigaChat API) для автоматической генерации тестовых заданий на основе произвольного текстового материала. Работа охватывает как теоретические аспекты, так и практическую реализацию проекта, включая архитектуру плагина, механизмы взаимодействия с внешним API, а также вопросы тестирования, устранения ошибок и оптимизации.

Основные выводы

1. Разработан функциональный прототип плагина, позволяющий преподавателю формировать тесты на основе текстового описания (промта) или загружаемого файла в формате .txt. Генерация происходит с помощью модели GigaChat, после чего создаётся XML-файл, совместимый с механизмами импорта в Moodle.
2. Интерфейс реализован в виде одностраничного веб-приложения, не требующего перезагрузки страницы, что значительно повышает удобство работы. Асинхронная отправка данных и обратная связь реализованы средствами JavaScript с использованием API fetch.
3. Серверная часть написана на PHP и включает в себя обработку авторизации через OAuth, взаимодействие с API GigaChat, парсинг полученного текста, преобразование данных в формат Moodle XML и возврат результата пользователю.
4. Особое внимание было уделено вопросам безопасности и надёжности: реализована изоляция конфиденциальных данных (client_id, secret), подключены SSL-сертификаты, предусмотрена обработка исключений и проверка валидности ответов API.

Плагин прошёл тестирование в различных режимах, включая генерацию из промта и на основе лекционных материалов. Результаты показали стабильную работу и корректное формирование структурированных XML-файлов, пригодных для загрузки в Moodle.

Перспективы дальнейшей разработки

Развитие и усовершенствование плагина может осуществляться в следующих направлениях:

- Поддержка других форматов входных данных (например, .docx, .pdf) с возможностью автоматической извлечения текста.
- Интеграция с системой ролей Moodle, позволяющая управлять доступом к плагину и назначать права по созданию и редактированию тестов.
- Реализация многоязычной поддержки, включая локализацию интерфейса и обработку исходных текстов на английском, французском и других языках.
- Добавление визуальных параметров генерации, таких как тип вопросов (множественный выбор, соответствие, ввод с клавиатуры), контроль числа вопросов и сложности.
- Хранение истории генерации, позволяющее пользователю возвращаться к ранее созданным тестам и повторно загружать XML-файлы.
- Интеграция с другими ИИ-моделями (в том числе GPT или других отечественных разработок), что обеспечит гибкость и независимость от конкретного API-провайдера.

Таким образом, в ходе проделанной работы, была достигнута поставленная цель – разработан плагин для Moodle для генерации тестовых заданий на основе ИИ. Представленная работа решает актуальную задачу автоматизации создания тестовых заданий в образовательной среде и демонстрирует потенциал применения искусственного интеллекта в педагогической практике. Полученные результаты могут быть использованы как в рамках учебного процесса, так и в исследовательской деятельности, направленной на изучение цифровых инструментов повышения качества образования.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Обзор системы электронного обучения Moodle. [Электронный ресурс] – Режим доступа: <https://beget.com/ru/kb/faq/moodle-review>
2. Основы ИИ. [Электронный ресурс] – Режим доступа: <https://habr.com/ru/articles/865664/>
3. Статья. Искусственный интеллект в образовании. [Электронный ресурс] – Режим доступа: <https://education.yandex.ru/knowledge/iskusstvennii-intellekt-v-obrazovanii-instrumenti-vizovi-i-perspektivi-dlia-studentov-i-prepodavatelei>
4. GigaChat API. [Электронный ресурс] – Режим доступа: <https://developers.sber.ru/docs/ru/gigachat/api/overview>
5. YandexGPT API. [Электронный ресурс] – Режим доступа: https://yandex.cloud/ru/docs/foundation-models/quickstart/yandexgpt?utm_referrer=https%3A%2F%2Fyandex.ru%2F
6. SberAI API. [Электронный ресурс] – Режим доступа: <https://developers.sber.ru/docs/ru/sber-api/overview>
7. Парсинг и обработка веб-страницы на PHP. [Электронный ресурс] – Режим доступа: <https://tproger.ru/digest/parse-html-via-php>
8. Официальный сайт PHP. [Электронный ресурс] – Режим доступа: <https://www.php.net/manual/ru/introduction.php>
9. Статья о PHP. [Электронный ресурс] – Режим доступа: <https://gb.ru/blog/php/>
10. Документация Python. [Электронный ресурс] – Режим доступа: <https://www.python.org/doc/>
11. XML. Documentation. [Электронный ресурс] – Режим доступа: <https://doka-guide.vercel.app/tools/xml/>
12. Справочник по HTML. [Электронный ресурс] – Режим доступа: <https://htmlbook.ru/html>
13. Documentation HTML. [Электронный ресурс] – Режим доступа: <https://html-doc.vercel.app/>
14. Документация JavaScript. [Электронный ресурс] – Режим доступа: <https://js-documentation.netlify.app/>

15. CSS. [Электронный ресурс] – Режим доступа: <https://doka.guide/css/>
16. Основы CSS. [Электронный ресурс] – Режим доступа: https://developer.mozilla.org/ru/docs/Learn_web_development/Getting_started/Your_first_website/Styling_the_content
17. Документация cURL. [Электронный ресурс] – Режим доступа: <https://curl.se/docs/>
18. UI – Что такое. [Электронный ресурс] – Режим доступа: <https://skyeng.ru/magazine/wiki/it-industriya/что-такое-ui/>
19. Введение в JavaScript. [Электронный ресурс] – Режим доступа: <https://learn.javascript.ru/intro>
20. Руководство по JavaScript. [Электронный ресурс] – Режим доступа: <https://habr.com/ru/companies/ruvds/articles/429552/>
21. Лутц М. Программирование на Python, том 1, 4-е издание. – Перевод с английского – СПб: Символ-плюс, 2011.
22. Documentation Python. [Электронный ресурс] – Режим доступа: <https://docs.python.org/3/index.html>
23. Статья. Что такое нейросети. [Электронный ресурс] – Режим доступа: <https://blog.eldorado.ru/publications/что-такое-нейросети-на-что-способны-как-работают-и-кому-нужны-35389>
24. Что такое XML. [Электронный ресурс] – Режим доступа: <https://www.unisender.com/ru/glossary/что-такое-xml/>
25. Бешенков С.А., Гридин В.А., Яшина Н.М. Современные информационные технологии в обучении // Вестник Московского университета. Серия 14: Психология. – 2015. № 1. – С. 114–126.
26. Тарасов А.Г. Применение нейронных сетей в образовательной среде // Научные ведомости Белгородского государственного университета. Серия: Экономика. Информатика. – 2019. Том 46, № 1. – С. 21–30.
27. Регулярные выражения в PHP. [Электронный ресурс] – Режим доступа: <https://codedokode.github.io/phpbook/11/regexp.html>

28. Кириченко А.Ф. Основы веб-дизайна и фронтенд-разработки. Учебное пособие. – СПб.: НИУ ИТМО, 2018. – 280 с.
29. Иванов В.В. Веб-разработка с нуля: полный цикл создания веб-проекта. Учебное пособие. – Самара: Самиздат, 2020. – 320 с.
30. Официальная документация Bootstrap Framework. [Электронный ресурс]. – Режим доступа: <https://getbootstrap.com/docs/5.1/getting-started/introduction/>
31. Бондаренко А.Л. Основы проектирования пользовательских интерфейсов для веб-приложений. – Харьков: ХПИ, 2019. – 240 с.
32. Шафранов-Куцев Г.Ф. Системы управления обучением и цифровые образовательные ресурсы. – М.: Академия, 2020. – 208 с.
33. JavaScript и взаимодействие с сервером: fetch, async/await. [Электронный ресурс] – Режим доступа: <https://learn.javascript.ru/fetch-api>
34. Как работать с API: советы разработчикам. [Электронный ресурс] – Режим доступа: <https://tproger.ru/translations/how-to-use-an-api>
35. Гаврилов Д.В. Применение искусственного интеллекта в системе образования // Современные проблемы науки и образования. – 2020. № 6. – С. 112–116.
36. Валиахметов С.Р. Практическое применение технологий HTML, CSS, JavaScript. Учебное пособие. – Казань: КГТУ, 2021. – 144 с.
37. Документация о безопасности Web-приложений. OWASP. [Электронный ресурс] – Режим доступа: <https://owasp.org/www-project-top-ten/>
38. Смирнов С.В. Интеллектуальные системы: искусственный интеллект и нейросетевые технологии. – СПб.: Питер, 2022. – 304 с.
39. Головина С.Н., Белякова Т.Г. Информационные технологии в образовании: учебное пособие. – М.: Академический проект, 2021. – 256 с.
40. Как работать с API. [Электронный ресурс] – Режим доступа: <https://tproger.ru/articles/chto-takoe-api-i-kak-s-nim-rabotat->

Листинг кода

index.html:

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Плагин ИИ</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <header class="top-bar">
    <button class="top-button">Плагин ИИ</button>
  </header>

  <div class="layout">
    <aside class="sidebar">
      <div class="sidebar-highlight"></div>
    </aside>

    <main class="main-content">
      <section class="card title-card">
        <h1>Плагин ИИ</h1>
      </section>

      <section class="card input-card">
        <form id="generate-form">
          <textarea placeholder="Введите промт для генерации тестов для
курса..."></textarea>
          <input type="file" name="textfile" accept=".txt" />
          <button class="generate-btn">Сгенерировать</button>
        </form>
      </section>

      <section class="card output-card">
        <h2>Сгенерированный текст:</h2>
        <div class="output-box">
          <p>.....</p>
          <p>.....</p>
        </div>
      </section>
    </main>
  </div>

  <script src="script.js"></script>
</body>
</html>

```

generate.php:

```
<?php
// Загрузка конфигурации
$config = require 'config.php';
$client_id = $config['client_id'];
$client_secret = $config['client_secret'];
$scope = $config['scope'];
$auth_key = base64_encode("$client_id:$client_secret");

// Получение токена
$ch = curl_init('https://ngw.devices.sberbank.ru:9443/api/v2/oauth');
curl_setopt_array($ch, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_POST => true,
    CURLOPT_HTTPHEADER => [
        "Authorization: Basic $auth_key",
        "RqUID: " . $client_secret,
        "Accept: application/json",
        "Content-Type: application/x-www-form-urlencoded"
    ],
    CURLOPT_POSTFIELDS => http_build_query(['scope' => $scope]),
    CURLOPT_SSL_VERIFYPEER => true,
    CURLOPT_SSL_VERIFYHOST => 2,
]);

$response = curl_exec($ch);
$data = json_decode($response, true);
$token = $data['access_token'] ?? null;
curl_close($ch);

if (!$token) {
    http_response_code(500);
    echo "Не удалось получить токен.";
    exit;
}

// Получение пользовательской инструкции и файла
$instruction = $_POST['prompt'] ?? '';
$context = '';

if (isset($_FILES['textfile']) && $_FILES['textfile']['tmp_name']) {
    $context = file_get_contents($_FILES['textfile']['tmp_name']);
}

if (!$instruction && !$context) {
    http_response_code(400);
    echo "Инструкция и файл отсутствуют";
    exit;
}
```

```

$prompt = $instruction;
if ($context) {
    $prompt .= "\n\nВот текст документа, на основе которого нужно составить
вопросы:\n$context";
}

// Отправка запроса в GigaChat
$headers = [
    "Authorization: Bearer $token",
    "Content-Type: application/json",
];

$body = json_encode([
    "model" => "GigaChat",
    "messages" => [[
        "role" => "user",
        "content" => $prompt
    ]],
    "temperature" => 0.8,
]);

$ch = curl_init('https://gigachat.devices.sberbank.ru/api/v1/chat/completions');
curl_setopt_array($ch, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_POST => true,
    CURLOPT_HTTPHEADER => $headers,
    CURLOPT_POSTFIELDS => $body,
    CURLOPT_SSL_VERIFYPEER => true,
    CURLOPT_SSL_VERIFYHOST => 2,
]);

$response = curl_exec($ch);
$data = json_decode($response, true);
$content = $data['choices'][0]['message']['content'] ?? 'Не удалось получить ответ';
curl_close($ch);

// === Отображение результата
echo "<pre>$content</pre>";

// === Парсинг теста с помощью регулярного выражения
$xml = new SimpleXMLElement('<quiz></quiz>');

$pattern = '/
    (?:(?:Вопрос\s*\d+[: : ]?\s*)          # заголовок "Вопрос 1:"
    (.*?)\s*[\r\n]+                        # текст вопроса

    (?:(?:[-\*]\s*)?a)\s*(.*?)\s*[\r\n]+   # вариант A
    (?:(?:[-\*]\s*)?b)\s*(.*?)\s*[\r\n]+   # вариант B
    (?:(?:[-\*]\s*)?c)\s*(.*?)\s*[\r\n]+   # вариант C
    (?:(?:[-\*]\s*)?d)\s*(.*?)\s*[\r\n]+   # вариант D

```

```

    (?:\*\*)?Ответ[: : ]?(?:\*\*)?\s*([a-d]) # жирный или обычный ответ
/isuX';
$content = preg_replace('/\*\*|###|^-\\s+/m', '', $content);

preg_match_all($pattern, $content, $matches, PREG_SET_ORDER);

if (!$matches) {
    echo "<p style='color:red;'>✗ Не удалось распознать структуру теста. Убедитесь, что
каждый вопрос оформлен строго по шаблону.</p>";
} else {
    foreach ($matches as $match) {
        $question = $xml->addChild('question');
        $question->addChild('text', htmlspecialchars(trim($match[1])));
        $options = ['a' => $match[2], 'b' => $match[3], 'c' => $match[4], 'd' =>
$match[5]];
        $correct_letter = strtolower($match[6]);

        foreach ($options as $letter => $text) {
            $option = $question->addChild('option', htmlspecialchars(trim($text)));
            $option->setAttribute('correct', $letter === $correct_letter ? 'true' :
'false');
        }
    }

    $xml_file = 'generated_test.xml';
    $xml->asXML($xml_file);

    echo "<a href='$xml_file' download>📄 Скачать XML</a>";
}
?>

```

scripts.js:

```

document.querySelector('#generate-form').addEventListener('submit', async (e) => {
    e.preventDefault();

    const form = e.target;
    const textarea = form.querySelector('textarea');
    const fileInput = form.querySelector('input[type="file"]');
    const outputBox = document.querySelector('.output-box');

    outputBox.innerHTML = '<p>Генерация...</p>';

    const formData = new FormData();
    formData.append('prompt', textarea.value);
    if (fileInput.files.length > 0) {
        formData.append('textfile', fileInput.files[0]);
    }
}

```

```

const response = await fetch('generate.php', {
  method: 'POST',
  body: formData
});

const result = await response.text();
outputBox.innerHTML = result;
});

```

style.css:

```

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: sans-serif;
}

body {
  background-color: #f5f5f5;
}

.top-bar {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  height: 60px;
  background-color: white;
  padding: 10px 20px;
  display: flex;
  justify-content: flex-end;
  align-items: center;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
  z-index: 1000;
}

.top-button {
  background-color: #004a99;
  color: white;
  padding: 8px 16px;
  border: none;
  border-radius: 20px;
  cursor: pointer;
  font-size: 1rem;
}

.layout {
  display: flex;
  margin-top: 60px;
}

```

```

    align-items: stretch;
}

.sidebar {
  width: 260px;
  background-color: white;
  margin: 20px;
  border-radius: 20px;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
  display: flex;
  flex-direction: column;
  overflow: hidden;
  height: auto;
}

.sidebar-highlight {
  height: 60px;
  background-color: #004a99;
  border-radius: 20px 20px 0 0;
}

.main-content {
  flex-grow: 1;
  padding: 30px;
  display: flex;
  flex-direction: column;
  gap: 20px;
  min-height: 100%;
}

.card {
  background-color: white;
  padding: 20px;
  border-radius: 20px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

.title-card h1 {
  font-size: 1.6rem;
  color: #333;
}

.input-card textarea {
  width: 100%;
  height: 170px;
  padding: 15px;
  border-radius: 10px;
  border: 1px solid #ccc;
  resize: none;
  font-size: 1rem;
}

```

```

    margin-bottom: 15px;
}

.generate-btn {
    background-color: #004a99;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 15px;
    font-size: 1rem;
    cursor: pointer;
}

.output-card h2 {
    margin-bottom: 10px;
    color: #444;
}

.output-box {
    background: #f9f9f9;
    padding: 15px;
    border-radius: 10px;
    font-family: monospace;
}

```

config.php:

```

<?php
// Конфигурация доступа к GigaChat API
return [
    'client_id' => 'd3176f64-62b9-4e3e-8af4',
    'client_secret' => '198a2bb0-0f20-4cd0-b2d',
    'scope' => 'GIGACHAT_API_PERS',
];
?>

```

Выпускная квалификационная работа выполнена мной совершенно самостоятельно. Все использованные в работе материалы и концепции из опубликованной научной литературы и других источников имеют ссылки на них.

« 23 » 06 2025 г.

А.В. Перошук (подпись) (Перошук А.В.) (Ф.И.О.)